Blocaïne

Manuel opérateur

(La solution open source pour l'automatisme)

Table des matières

- 1 - Généralité....3
  - 1.1 - Caractéristiques....4
  - 1.2 - Avancement du projet....4
  - 1.3 - Précautions d'usages, responsabilités....4
- 2 - Principes....5
  - 2.1 - Tout est bloc....5
  - 2.2 - Chargement à la volée (HotSwap)....5
  - 2.3 - Méthode d'exécution....5
  - 2.4 - Typage des variables....5
  - 2.5 - Bit de validité....5
  - 2.6 - Valeurs par défaut....6
- 3 - Matériel....7
  - 3.1 - OS et « temps réel »....7
  - 3.2 - Matériel pour évaluation....7
  - 3.3 - Matériel pour application standard....7
    - 3.3.1 - PC de développement....7
    - 3.3.2 - Machine cible....7
- 4 - L'Éditeur de blocs....8
  - 4.1 - Généralités....8
  - 4.2 - Interface graphique....9
    - 4.2.1 - Barre de titre....10
    - 4.2.2 - Barre de menu....10
      - 4.2.2.a - Menu « Bloc »....10
      - 4.2.2.b - Menu « Target »....11
      - 4.2.2.c - Menu « Edit »....11
      - 4.2.2.d - Menu « Help »....12
    - 4.2.3 - Icônes....13
    - 4.2.4 - Zone Graphique....14
      - 4.2.4.a - Navigation....14
      - 4.2.4.b - Menus contextuels....15
        4.2.4.b.1 Menu (par défaut)....15
        4.2.4.b.2 Menu « entête »....15
        4.2.4.b.3 Menu « Entrée/Sortie »....16
      - 4.2.4.c - Insérer d'un bloc....17
      - 4.2.4.d - Supprimer d'un bloc....18
      - 4.2.4.e - Créer un lien....18
      - 4.2.4.f - Supprimer un lien....19
    - 4.2.5 - Barre de status....20

- 4.3 - Les entrées/sorties....20
- 4.3.1 - bloc <input>....20
- 4.3.2 - bloc <output>....21
- 4.3.3 - Interface externe d'un bloc user....22
- 4.4 - Mode : Editing / Monitoring....23
- 4.4.1 - Mode : édition (Editing)....24
- 4.4.2 - Mode : visualisation dynamique (Monitoring)....25
- 4.5 - Bit de validité....25
- 5 - L'Exécuteur....26
- 5.1 - Généralités....26
- 5.2 - Méthode d'exécution....26
- 5.3 - Chargement à la volée (hotswap)....26
- 5.4 - Bit de validité....27
- 5.5 - Interface IHM/SCADA....27
- 5.6 - Entrées/Sorties déportées....27
- 5.7 - Raspberry pi GPIO....27
- 5.8 - Créer un nouveau bloc system....28
- 5.8.1 - Créer l'interface externe....28
- 5.8.2 - Créer le code Python....29
- 5.8.2.a - Créer la procédure Python liée au nouveau bloc system....29
- 5.8.2.b - Associer la procédure au paramètre « nom de bloc »....31
- 5.8.2.c - Définir le nom de fichier du nouveau bloc system....32
- 5.9 - Startup (autorun)....33
- 5.9.1 - Ajouter un bloc dans le « startup »....33
- 5.9.2 - Retirer un bloc du « startup »....33
- 5.9.3 - Supprimer définitivement un bloc....33
- 5.10 - Serveur Web....34
- 5.10.1 - Généralités....34
- 5.10.2 - Adresse du serveur web....34
- 5.10.3 - Menu principal....35
- 5.10.4 - Exemples de page web....36
- 5.10.4.a - Page « Bloc »....36
- 5.10.4.b - Page « Output »....37
- 5.10.4.c - Page « Overriding »....38
- 5.10.4.d - Page « Task & loads »....39
- 5.10.4.e - Page « Startup »....40
- 5.10.4.f - Page « Connection »....41
- 5.10.4.g - Page « Debug »....42

- 1 - Généralité

Blocaïne est une plateforme logicielle conçue pour l'automatisation des procédés industriels. Elle comprend deux composants principaux :

- Un **Éditeur de blocs**, qui permet de créer des programmes en assemblant des blocs fonctionnels.
- Un **Exécuteur ,** qui exécute les programmes créés avec l'**Éditeur de blocs,** pilotant ainsi l'équipement industriel via des entrées/sorties déportées.

L'**Éditeur de blocs** est installé sur un « PC de Développement » et communique par Ethernet avec l'**Exécuteur**, déployé sur un autre ordinateur appelé « Machine cible ».

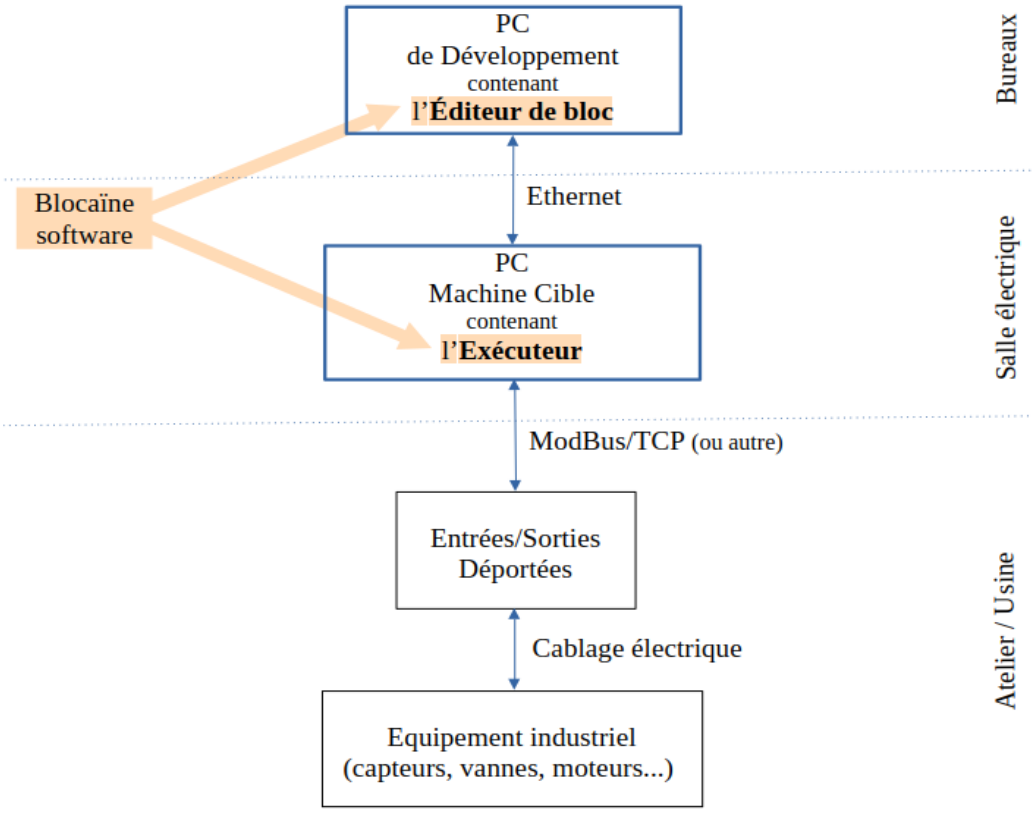


Figure 1 : Architecture générale

La connexion Ethernet permet de transférer les programmes créer avec l'**Éditeur de blocs** vers l'**Exécuteur**, elle permet également la visualisation en temps réel au sein de l'**Éditeur de blocs** des variables des programmes en cours d'exécution par l'**Exécuteur**.

- 1.1 - Caractéristiques

- « Chargement à Chaud » (**HotSwap**) : Une nouvelle version d'un programme peut être chargée alors que l'ancienne version est en cours d'exécution.
- Programmation graphique sous forme de blocs fonctionnels chaînés
- Chaque variable possède nativement un bit de validité, qui se propage de bloc en bloc.
- Typage dynamique des variables.
- L'ordre d'exécution des blocs est géré automatiquement
- La mise en page des blocs est réalisée automatiquement
- Un serveur web permet de connaître l'état de la Machine Cible
- Un serveur OPC UA permet l'interfaçage avec les IHM/SCADA
- Le protocole ModBus/TCP permet l'interfaçage avec les E/S déportées
- Blocaïne est multiplateforme (Linux, Windows, macOS...)
- Blocaïne est libre de droits et en sources ouvertes

- 1.2 - Avancement du projet

Le projet Blocaïne en est au stade de prototype Fonctionnel.

Il est téléchargeable pour évaluation sur « github.com », la procédure d'installation est décrite dans le document « evaluation.pdf »

Pour tous renseignements, vous pouvez me contracter par Email : blocaine@barachet.com

- 1.3 - Précautions d'usages, responsabilités

L'utilisation de Blocaïne se fait sous la responsabilité de l'utilisateur.

- 2 - Principes

- 2.1 - Tout est bloc

Chaque bloc est constitué d'entré(s) de sortie(s) et du code permettant d'évaluer les sorties en fonction des entrées et parfois des sorties .

- Pour les blocs de la bibliothèque de base (appelés blocs **System**) c'est une fonction écrite en Python qui évalue les sorties en fonction des entrées
- Pour les blocs créés par l'utilisateur (blocs **user**) c'est un chaînage de blocs **system** ou **user**, qui permet d'évaluer les sorties en fonction des entrées

Un programme exécutable n'est autre qu'un bloc **user**, Un sous-programme se présente aussi sous la forme d'un bloc **user**, les entrées/sorties des blocs sont eux même des blocs (<inputs>, <output>), le bloc <input> possède une sortie définie par un bloc <output>, tandis que le bloc <output> possède une sortie définie par un bloc <input>. Bref tous est bloc.

- 2.2 - Chargement à la volée (HotSwap)

l'**Exécuteur** dispose pour chaque programme (bloc **user**) de deux versions **Shift A** et **Shift B**. La nouvelle version d'un bloc est toujours téléchargée dans le **Shift** qui n'est pas en cours d'exécution, lorsqu'un **HotSwap** est initié, les valeurs courantes des variables mémorisées du **Shift** en cours d'exécution sont transférés à l'autre **Shift,** qui prend alors la relève.

- 2.3 - Méthode d'exécution

Pour chaque bloc **user** en cours d'exécution (Running), l'**Exécuteur** évalue périodiquement tous les blocs <output> associés à une tache périodique. Chaque entrée d'un bloc <output> est évalué par l'**Exécuteur** en exécutant d'abord le bloc précédent, et ainsi de suite.

Cette approche élimine la nécessité de définir explicitement l'ordre d'exécution des blocs.

- 2.4 - Typage des variables

Puisque Blocaïne s'appuie sur le langage Python, les variables (d'entrée et de sortie) utilisées par les blocs sont typées dynamiquement, leur type peut être quelconque y compris des structures complexes, dés lors qu'il s'agit d'une combinaison de types reconnue par Python (chaîne de caractères, entier, flotant, liste, dictionnaire, etc.).

- 2.5 - Bit de validité

Un bit de validité associé à chaque sortie de bloc est évalué automatiquement à chaque exécution en fonction de la validité des entrées et de la possibilité ou non d'évaluer de la sortie.

- 2.6 - Valeurs par défaut

Chaque entrée de bloc, qu'il s'agisse d'un bloc **system** ou **user**, possède une valeur par défaut modifiable à l'instanciation.

- 3 - Matériel

Étant donné que Blocaïne est uniquement composer de 2 programmes en Python (un pour l'**Exécuteur** et l'autre pour l'**Éditeur de blocs**), il suffit que le matériel puisse exécuter du code Python (version 3.6 ou supérieur) et qu'il dispose d'un adaptateur réseau (connexion internet) WIFI ou filaire pour que Blocaïne puisse fonctionner.

- 3.1 - OS et « temps réel »

Tous les systèmes d'exploitation sont à priori compatibles (Windows, Linux, macOS…), mais leurs performances « temps réel » ne sont pas identiques.

Seulement l'**Exécuteur** est soumis à des contraintes temps réel, l'**Éditeur de blocs** qui ne sert que d'interface opérateur, peut accueillir n'importe quel OS.

Vous pouvez apprécier la stabilité des tâches périodiques via le serveur web fourni par la cible (l'**Exécuteur**) en comparant depuis la page « Task & Load » la période théorique avec les temps de cycle min et maxi. Après de nombreux essais on peut dire qu'un OS Linux « classique » peut convenir dans la plupart des cas, mais si les contraintes temps réel sont exigeantes il faudra basculer vers Linux-RT ou autre.

- 3.2 - Matériel pour évaluation

Un simple PC Windows, PC linux ou Mac est tout à fait adapté, puisque La version téléchargeable sur Github.com (voir procédure d'évaluation) est configurée pour faire tourner l'**Exécuteur** et l'**Éditeur de blocs** sur la même machine.

- 3.3 - Matériel pour application standard

- 3.3.1 - PC de développement

De nos jour le langage Python est présent nativement sur « tous » les PC & Mac, comme l'**Éditeur de blocs** n'utilise que les modules standards de Python, l'installation l'**Éditeur de blocs** se résumera à une simple copie de fichier. Votre PC portable favori est donc tout à fait adapté pour faire tourner l'**Éditeur de blocs**, si la version de Python est 3.6 ou supérieur.

- 3.3.2 - Machine cible

Le **Raspberry pi 5** semble être le meilleur choix Pour faire tourner l'**Exécuteur** pour les raisons suivantes :

- son prix : une cinquantaine d'euros en version 1Go de RAM
- Ses performances en simple cœur grâce à son Arm Cortex-A76
- La simplicité de mise en œuvre, en téléchargeant une image ISO de la carte SD, cela vous épargnera l'installation des modules Python non standards

- 4 - L’Éditeur de blocs

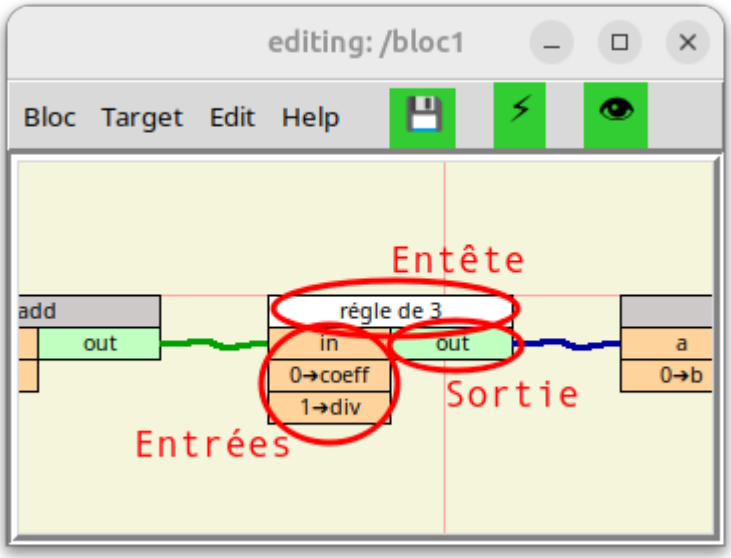
- 4.1 - Généralités

L’**Éditeur de blocs** permet :

- La création et la modification de programmes sous forme de blocs **user** contenant des blocs fonctionnels chaînés entre eux.
- La compilation , le téléchargement et le lancement de l’exécution du bloc **user** en cours d’édition dans la « Machine Cible ».
- la visualisation dynamique des entrées/sorties des blocs en cours d’exécution.
- Forçage des entrées/sorties des blocs en cours d’exécution.

Chaque bloc est constitué de :

- Une entête : contenant le nom du bloc
- D’entrée(s) : située(s) à gauche sous l’entête, contenant le nom de l’entrée
- De sortie(s) : située(s) à droite sous l’entête à, contenant le nom de la sortie



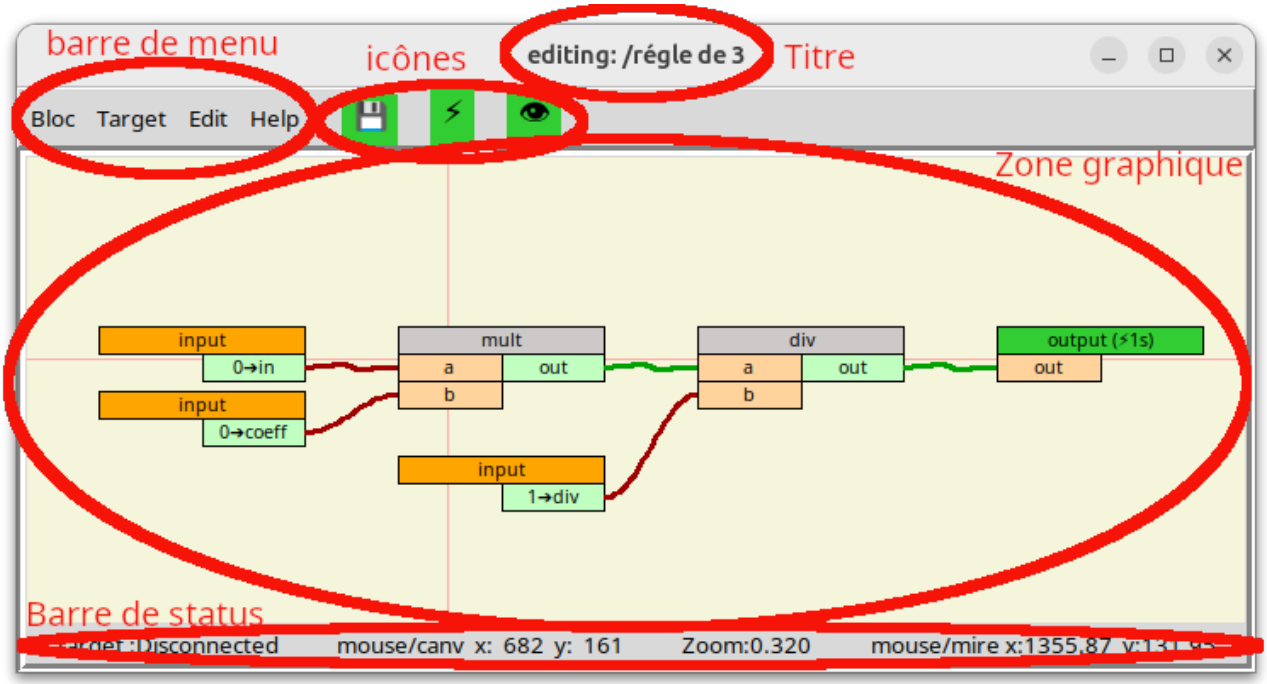
Les blocs peuvent êtres chaînés entre eux par des liens, notez qu’une sortie ne peut être liée qu’à une entrée et qu’une entrée ne peut être liée qu’à une seule sortie.

À chaque bloc correspond un fichier nommé : « *nom du bloc*.bloc ». Les fichiers des blocs **system** sont dans le répertoire /blocs/system, alors que les fichiers des blocs **user** sont dans /blocs/user.

- 4.2 - Interface graphique

L'interface graphique de l'**Éditeur de blocs** « Blocaïne » est constituée d'une ou plusieurs fenêtres, chacune contient les éléments suivants :

1. Une barre de titre (de la fenêtre)
2. Une barre de menu
3. Trois icônes : 💾, ⚡, 👁️
4. Une zone graphique, ou vous déposez et reliez entre eux des blocs, afin de constitué un programme (blocs **user**).
5. Une barre de status :



Interface graphique de l'éditeur de bloc

- 4.2.1 - Barre de titre

En haut de chaque fenêtre se situe une barre de titre qui contient :

- Le mode en cours : **Editing** ou **Monitoring**
- Le nom du bloc en cours d'édition ou de débogage

Notez que si le bloc a été ouvert en temps que sous-bloc, sous-sous-bloc etc, le chemin ayant permis son ouverture apparaîtra devant le nom du bloc. Cela est utile en mode **Monitoring (**débogage) pour déterminer à quelle instance du bloc appartiennent les variables affichées dans cette fenêtre.

- 4.2.2 - Barre de menu

La barre de menu située en haut à gauche sous la barre de titre, contient les quatre menus suivants :

- **Bloc** : Est équivalent au menu « Fichier » de la plupart des logiciels.
- **Target** : Gère l'interface avec la « Machine Cible » (donc avec l'**Exécuteur).**
- **Edit** : Gère la présentation.
- **Help** : Affiche la documentation.

- 4.2.2.a - Menu « Bloc »

- **Open** : Permet d'ouvrir un fichier contenant un bloc.
- **Save [F3]** : Permet de sauvegarder le bloc en cours d'édition dans un fichier dont le nom correspond au nom du bloc courant.
- **Save as [F4]** : Permet de sauvegarder le bloc en cours d'édition dans un fichier dont le nom sera saisie par l'utilisateur.
- **Update [F5]** : Permet de mette à jour toutes les interface externe des blocs **user** utilisés comme sous-programme dans le bloc **user** en cour d'édition.
- **Change Properties** : Permet de modifier les champs associés à l'entête du bloc **user** en cours d'édition comme « l'auteur du bloc », « les mots clef »...mais aussi son « nom ».
- **Open new windows** : Permet d'ouvrir un nouvel **Éditeur de bloc** dans une fenêtre distincte, entièrement indépendant de l' **Éditeur de bloc** d'origine.
- **Quit [Escape]** : Ferme la fenêtre courante, sans sauvegarder le bloc **user** en cours d'édition.

- 4.2.2.b - Menu « Target »

- **Disconnect from Target** ou **Connect to Target :** Permet de se connecter ou se déconnecter de la « Machine Cible » (donc de l'**Exécuteur**).
- **Open Target's web pages** ***xxx.xxx.xxxx.xxxx:xxxx :*** Ouvre dans votre navigateur internet la page web principale fournie par le serveur HTTP de la cible.
- **Check <*nom_du_bloc*>** : Vérifie si le bloc en cours d'édition est valide et compilable.
- **Check & Transfert <*nom_du_bloc*>** : Vérifie si le bloc en cours d'édition est valide le compile, puis transfert le résultat dans la « machine Cible » vers le **Shift** qui n'est pas « running ».
- **Check, Transfert & Execute <*nom_du_bloc*>** : Vérifie si le bloc en cours d'édition est valide, le compile, transfert le résultat dans la « Machine Cible » vers un **Shift** qui n'est pas « running », puis l'**Exécuteur** lance son exécution: dans le cas ou le bloc est déjà en cours d'exécution sur l'autre **Shift** un chargement à chaud (HotSwap) sera initié, sinon un simple démarrage (Run) sera initié.
- **Monitoring (Start/Stop) [Space]** : Permet de basculer du mode édition au mode débogage.

- 4.2.2.c - Menu « Edit »

- **Auto layout [F7] :** Cette commande effectue une mis en page automatique.
- **Status bar (show/hide) :** Cette commande permet de montrer ou cacher la « barre de status ».

- 4.2.2.d - Menu « Help »

- **Versions** : Donne la version de l'**Éditeur de blocs** et de la structure des blocs.
- **Mouse usage** : Donne des instructions sur l'utilisation de la souris :
  - [Left click] to select, to use menu
  - [Left held down] to scroll within window, to move bloc, to link I/O bloc
  - [Left double-click] to open **user** bloc
  - [Right button] to open the context menu
  - [wheel button] to monitor the target variables (on/off)
  - [wheel rotation] to zoom (in/out)
- **Key usage** : Donne des instructions sur l'utilisation du clavier :
  - [F1] Open the documentation for the bloc whose header is located under the cursor
  - [F3] to save curant bloc
  - [F4] to save curant bloc as
  - [F5] to update curant bloc (mise à jour des interfaces des sous-blocs)
  - [F7] to layout curant bloc
  - [Delete] Delete the bloc whose header is located under the cursor
  - [Space] to close current **Block Editor** window
  - [Escape] to close window
- **General Documentaion** : Ouvre le présent document.

- 4.2.3 - Icônes

Ces icônes permettent un accès rapide au fonctionnalités les plus fréquemment utilisées :

- "💾" : sauvegarde le bloc courant.
- "⚡" : vérifie la cohérence du bloc courant, le convertit (compilation) dans un format compréhensible par l'**Exécuteur**, transfert le résultat dans la Machine Cible puis demande à l'**Exécuteur** :
    1. Son démarrage (Run), si le bloc courant n'est pas déjà en cours d'exécution dans la machine Cible.
    2. Son chargement à chaud (HotSwap), si le bloc courant est déjà en cours d'exécution dans la machine Cible.
- "👁️" : bascule d'un mode à l'autre entre Editing et Monitoring.

- 4.2.4 - Zone Graphique

Cette zone vous permet de construire un programme (bloc **user**) en y déposant des blocs **system** ou **user** et en les reliant entre eux.

- 4.2.4.a - Navigation

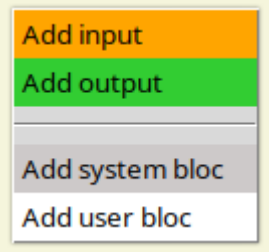
Pour naviguer dans cette zone, il faut utiliser une souris :

- **Rotation de la molette** : permet de zoomer/dézoomer.
- **Déplacement de la souris avec bouton gauche enfoncée** :
  - **Sur une zone vierge** : permet de déplacer la zone visible (scroller).
  - **Sur une E/S** : permet de créer un lien entre une entrée et une sortie.
  - **Sur une entête de bloc** : permet le déplacement du bloc.
- **Clic droit** : permet de faire apparaître le menu contextuel.
- **Double clic gauche** :
  - S**ur l'entête d'un bloc user :** ouvre le bloc **user** pointé dans une nouvelle fenêtre.
  - **Sur une E/S d'un bloc user :** ouvre le bloc **user** de l'I/O pointée dans une nouvelle fenêtre et positionne l'entrée ou la sortie au centre de la zone graphique.
- **Clic gauche** : permet de sélectionner un champ dans un menu.
- **Clic sur le bouton de la molette** : active/déactive la visualisation en temps réel des valeurs des variables situées dans la cible (Monitoring On/Off).

- 4.2.4.b - Menus contextuels

4.2.4.b.1 Menu (par défaut)

Un clic droit sur une zone vierge, fait apparaître le menu contextuel suivant :

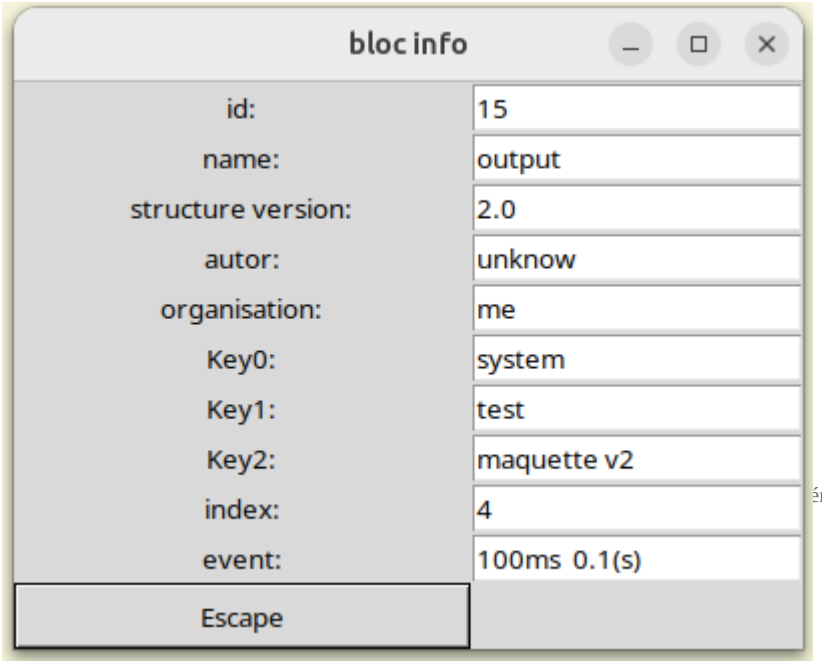


Ce menu permet d'ajouter un bloc.

4.2.4.b.2 Menu « entête »

Un clic droit sur l'entête d'un bloc, fait apparaître le menu contextuel suivant :

1. **Delete** : permet la suppression du bloc
2. **Update** : met à jour l'interface externe du bloc, à partir du fichier de ce bloc.
3. **Change event** : permet l'association le bloc à une tache périodique afin qu'il soi exécutable. Ce champ n'est disponible que pour les blocs <output>
4. **Position :2/3**, **move up**, **move down** : permet de modifier l'ordre des entrées ou des sorties dans l'interface externe du bloc **user** en cours d'édition. Ce champ n'est disponible que pour les blocs <input> ou <output>.
5. **Documentation** : ouvre le fichier d'aide du bloc dans le navigateur web par défaut. Ce champ n'est disponible que si un fichier d'aide nommé « bloc_ *{type de bloc}*_{*nom du bloc}*.html » est présent dans le répertoire « /Documentation ».
    1. *type de bloc* : peut être ; **user** ou **system**
    2. *nom du bloc* **:** correspondant au nom défini dans l'entête du bloc
6. **Properties** : afficher les propriétés du bloc comme ceci.

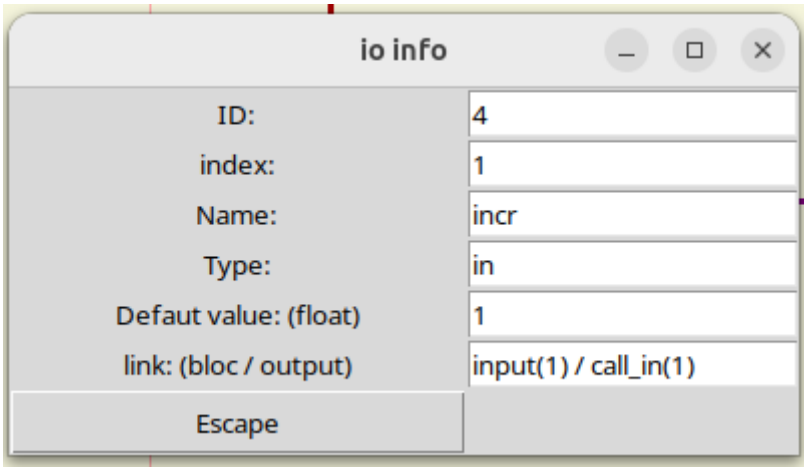


4.2.4.b.3 Menu « Entrée/Sortie »

Un clic droit sur une entrée ou un sortie d'un bloc, fait apparaître le menu contextuel suivant :

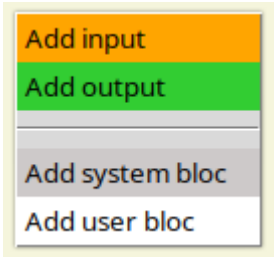
1. **Delete link** : Permet la suppression du lien avec le bloc précédent. Ce champ n'est disponible que si vous avez cliqué sur une entrée liée à une sortie d'un autre bloc.
2. **Rename :** Permet de renommer une entrée ou une sortie, ce nouveau nom apparaîtra dans l'interface externe du bloc. Ce champ n'est disponible que pour les blocs <input> et <output>.
3. **Change comment :** Permet de modifier le commentaire d'une entrée ou d'une sortie, ce nouveau commentaire apparaîtra dans l'interface externe du bloc. Ce champ n'est disponible que pour les blocs <input> et <output>.
4. **Set default value** : Permet de donner une valeur et un type par défaut à une entrée. S'il s'agit d'un bloc <input> cette valeur par défaut apparaîtra dans l'interface externe du bloc, sinon cette valeur par défaut est associée à l'instance du bloc.
5. **Open** : Permet d'ouvrir une nouvelle fenêtre qui édite le bloc **user** de l'entrée, et positionne l'entrée au centre de la zone graphique.
6. **Create locale name** : Permet de définir un nom associé à l'instance du bloc. Ce champ est disponible pour tous les blocs sauf ; <input> et <output>.
7. **Delete locale name** : Permet de supprimer le nom associé à l'instance du bloc. Ce champ est disponible pour tous les blocs sauf ; <input> et <output>.
8. **Create locale comment** : Permet de définir un commentaire associé à l'instance du bloc. Ce champ est disponible pour tous les blocs sauf ; <input> et <output>.
9. **Delete locale comment** : Permet de supprimer le commentaire associé à l'instance du bloc. Ce champ est disponible pour tous les blocs sauf ; <input> et <output>.
10. **Add Memory** : Permet de définir une sortie en temps que sortie « mémorisée ». Ce champ n'est disponible que pour les sorties d'un bloc **system,** c'est à dire pour l'entrée d'un bloc <output> lorsque le bloc en cours d'édition est un bloc **system**.
11. **Delete Memory** : Permet de supprimer la « mémorisation » d'un sortie. Ce champ n'est disponible que pour les sorties d'un bloc **system,** c'est à dire pour l'entrée d'un bloc <output> lorsque le bloc en cours d'édition est un bloc **system**.
12. **Add initial value** : Permet de assigner une valeur initiale associé à l'instance du bloc à une sortie « mémorisée ». Ce champ n'est disponible que pour les sorties « mémorisées », c'est à dire pour une entrée déjà « mémorisée » d'un bloc <output>.
13. **Change initial value** : Permet de modifier une valeur initiale à une sortie « mémorisée ». Ce champ n'est disponible que pour les sorties « mémorisées », c'est à dire pour une entrée déjà « mémorisée » d'un bloc <output>.

14. **Delete initial value** : Permet de supprimer une valeur initiale à une sortie « mémorisée ». Ce champ n'est disponible que pour les sorties « mémorisées », c'est à dire pour une entrée déjà « mémorisée » d'un bloc <output>.
15. **Override** : Permet de surcharger le type et la valeur d'une entrée ou d'une sortie dans la Machine Cible. Ce champ n'est disponible que si le mode « Monitoring » est activé.
16. **Properties** : Afficher les propriétés de l'entrée ou de la sortie comme ceci.



- 4.2.4.c - Insérer d'un bloc

Pour insérer un bloc, il faut faire un clic droit sur une zone vierge, afin de faire apparaître le menu contextuel suivant :

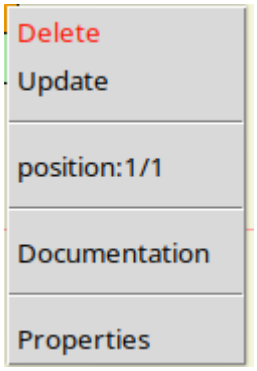


Il faut alors choisir le type de bloc que vous souhaitez :

- Add input : pour créer une nouvelle entrée dans le bloc **user** en cours d'édition.
- Add output : pour créer une nouvelle sortie dans le bloc **user** en cours d'édition.
- Add system bloc : pour insérer un bloc faisant partie de la bibliothèque standard.
- Add user bloc : pour insérer un bloc **user** précédemment créé.

- 4.2.4.d - Supprimer d'un bloc

Pour supprimer un bloc, vous pouvez faire un clic droit sur l'entête du bloc à supprimer, afin de faire apparaître le menu contextuel suivant :



puis sélectionner Detete.

Vous pouvez aussi positionner le curseur sur l'entête du bloc à supprimer, puis appuyer sur la touche « Suppr ».

- 4.2.4.e - Créer un lien

Pour lier la sortie d'un bloc à l'entrée d'un autre bloc, enfoncez le bouton gauche de la sourie sur la sortie du premier bloc, puis glissez (bouton gauche toujours enfoncé) jusqu'à l'entrée du second bloc puis relâcher le bouton gauche. Les liens peuvent aussi être créés indifféremment d'entrée vers sortie ou de sortie vers entrée, mais une entrée ne peut avoir qu'une seule source (donc provenir d'une seule sortie).

- 4.2.4.f - Supprimer un lien

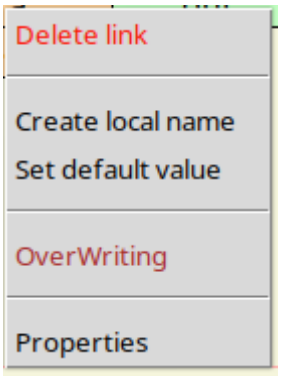
Il y deux possibilités pour supprimer un lien :

Soit faire un clic droit sur le lien, afin de faire apparaître le menu contextuel suivant :



puis sélectionner Delete link

Soit faire un clic droit sur l'entrée située à l'extrémité du lien, afin de faire apparaître le menu contextuel suivant :



puis sélectionner Delete link

- 4.2.5 - Barre de status

Elle indique :

- l'état de la connexion avec la Machine Cible.
- la position de la souris par rapport à la zone graphique (canvas).
- le coefficient de zoom courant.
- la position de la souris par rapport à la mire.

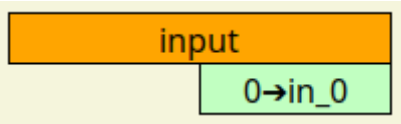
- 4.3 - Les entrées/sorties

L'interface d'un bloc **user** est constitué ; d'entrées et de sorties, celles-ci sont optionnelles mais indispensable pour l'interconnexion avec les autres blocs.

- Une entrée est collectée à l'aide du bloc **system** nommé <input>
- Une sorties est délivrée à l'aide du bloc **system** nommé <output>

- 4.3.1 - bloc <input>

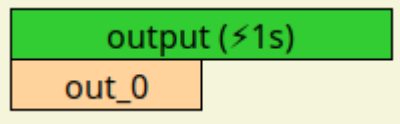
Voici l'interface externe du bloc **system** <input>



Sa fonction est de définir une entrée d'un bloc **user**. Le bloc <input> n'a pas d'entrée, mais possède une sortie nommée « in_0 », c'est une variable (de type quelconque) qui rentrera dans du bloc **user,** sa valeur par défaut est 0 (int), elle peut être ajustée pour chaque instance d'un bloc <input>. Cette sortie peut être renommée, le nouveau nom apparaîtra dans l'interface externe du bloc **user** en tant qu'entrée.

- 4.3.2 - bloc <output>

Voici l'interface graphique du bloc **system** <output>



Sa fonction est de définir une sortie d'un bloc **user**. Le bloc <output> n'a pas de sortie, mais possède une entrée nommée « out_0 », c'est une variable (de type quelconque) qui sortira du bloc **user,** elle n'a pas de valeur par défaut, mais une valeur par défaut peut être ajouter pour chaque instance d'un bloc <output>. Cette entrée peut être renommée, le nouveau nom apparaîtra dans l'interface externe du bloc **user** en tant que sortie.

- 4.3.3 - Interface externe d'un bloc user

Voici un bloc **user** nommé <règle de 3> qui contient trois blocs <input> et un bloc <output>.

- la sortie du première bloc <input> a été renommée en « in »
- la sortie du second bloc <input> a été renommée en « coeff »
- la sortie du troisième bloc <input> a été renommée en « div »
- l'entrée du bloc <output> a été renommée en « out »

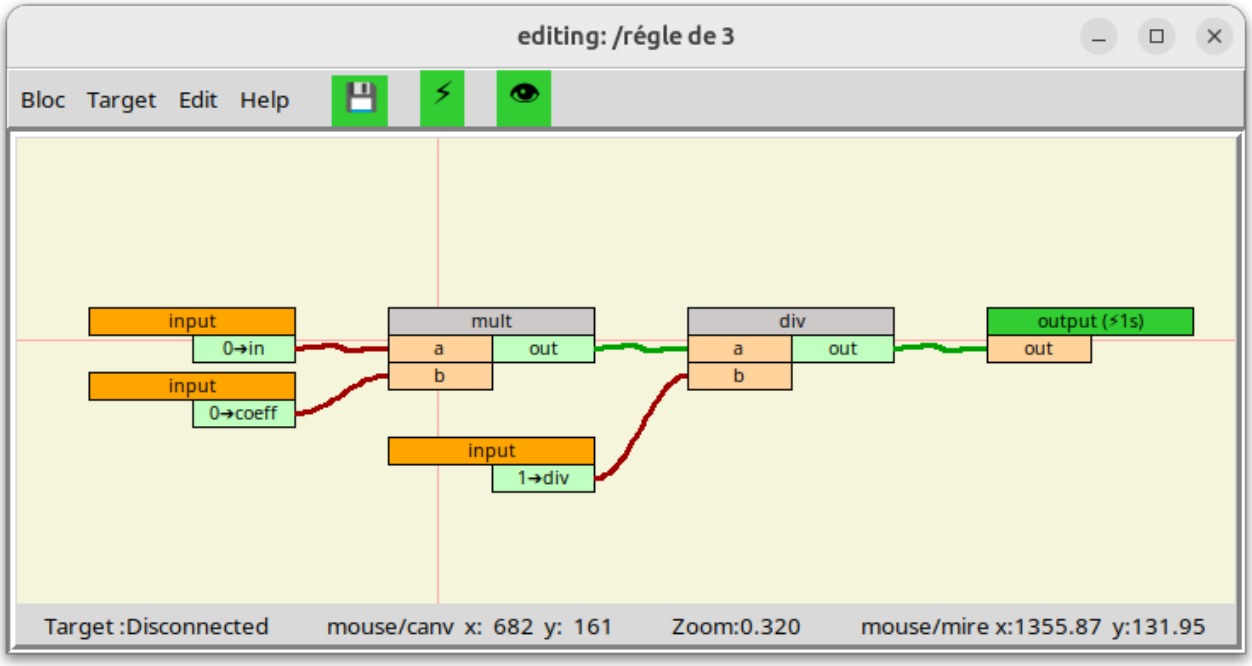


Figure : bloc **user** <règle de 3 >

Voici l'interface externe du bloc « règle de 3 », ici un bloc **user** nommé <bloc1> fait référence au bloc « régle de 3 », les 3 entrées et la sortie correspondent aux <input>/<output> définies dans le bloc « régle de 3 » sont bien présentes dans son interface externe.

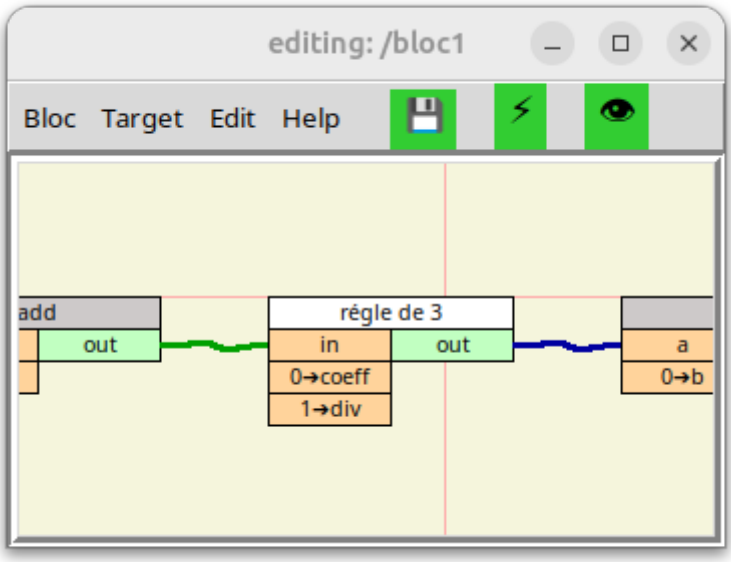


Figure ; interface externe du bloc **user** <régle de 3>

L'ordre d'apparition des entrées et des sorties dans l'interface externe d'un bloc **user** peut être modifier de la façon suivante :

1. Ouvrir le bloc **user** dont vous voulez modifier l'ordre des entrées et/ou des sorties de son interface externe, en double cliquant sur l'entête du bloc à modifier, si il est présent dans la zone graphique, ou en utilisant la barre de menu (bloc/open) ou (bloc/open new window puis bloc/open).
2. Faites un clic droit sur l'entête du bloc <input> ou <output> selon que vous souhaitiez modifier l'ordre des entrées ou des sorties. Afin de faire apparaître le menu contextuel. Dans ce menu il y a un champs **Position:x/y** où « x » indique la position de l'entrée ou la sortie que vous avez sélectionnée dans l'interface externe du bloc, « y » indique le nombre d'entrée ou de sortie du bloc.
3. Vous pouvez alors sélection **move up** ou **move down.** pour la décaler d'une position vers haut ou vers le bas.

- 4.4 - Mode : Editing / Monitoring

Le basculement du mode « édition » au mode « visualisation dynamique » et vice versa, s'effectue soit en cliquant sur l'icône « œil » (👁), soit en appuyant sur le bouton de la molette de la souris, bien sûr le bloc édité doit être en cours d'exécution (Running) dans la Machine Cible pour pouvoir basculé en mode « visu dynamique ».

- 4.4.1 - Mode : édition (Editing)

Ce mode est dédié à création et à la modification de blocs **user**.

Voici un exemple de bloc **user** qui calcule la moyenne de 2 nombres en mode « Editing », notez que le titre de la fenêtre indique le mode en cours d'utilisation:

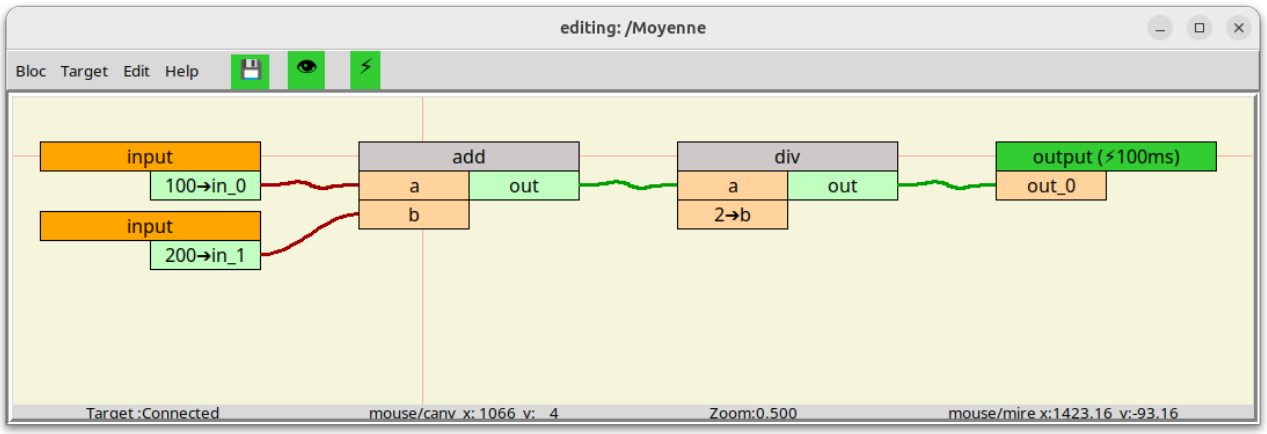


Figure 2 : bloc en mode édition

- 4.4.2 - Mode : visualisation dynamique (Monitoring)

Ce mode est dédié au débogage, il permet la visualisation en temps réel des valeurs des variables d'un bloc **user** ou d'un sous-bloc **user...**

la validité de chaque variable est représentée par la couleur de fond :

- Vert = valide
- Rouge = invalide
- jaune = forcé et valide
- Noir = Forcé et invalide

Voici un exemple du bloc <moyenne> en mode « Monitoring » (en visualisation dynamique), notez que le titre de la fenêtre indique le mode en cours d'utilisation:

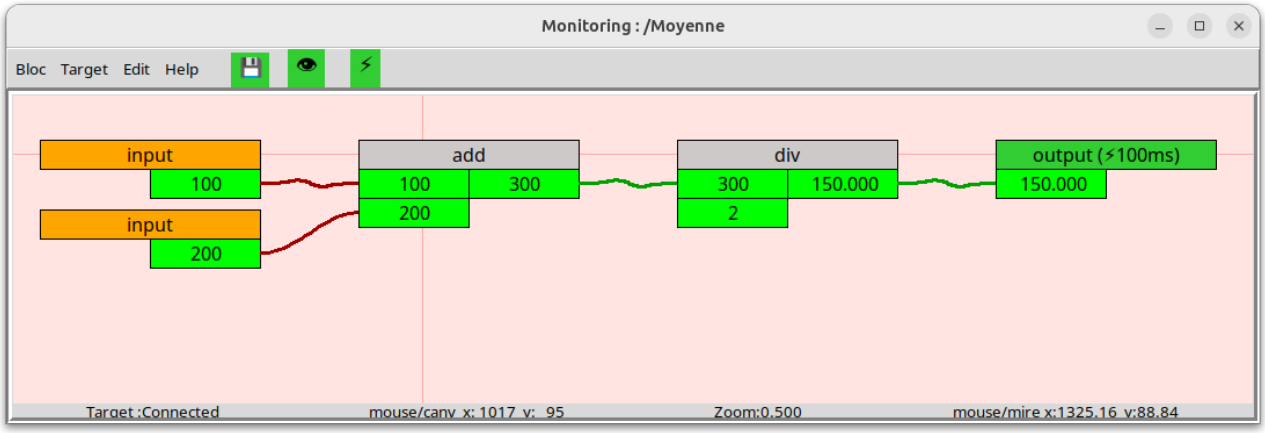


Figure 3 : bloc en mode visu dynamique

- 4.5 - Bit de validité

Les bits de validité peuvent être lus ou modifiés en utilisant les blocs **system** suivants :

- <valideRead> : permet de lire le bit de validité, cela permet par exemple de sélectionner une position de replie dans le cas de variables « invalide »
- <valideWrite> : permet l'écriture du bit de validité, autorisant l'utilisateur à définir sa propre équation de validité.

- 5 - L’Exécuteur

- 5.1 - Généralités

Rappel : L’**exécuteur** est installé sur un PC appelé « Machine cible », il exécute les programmes (blocs **user**) qui pilotent l’équipement industriel en utilisant les entrées/sorties déportées.

Certaines commandes liées aux blocs **user** déjà présent dans l’**exécuteur**, telles que : le démarrage, l’arrêt, l’initialisation, le swap, la suppression, peuvent être accomplies par l’**exécuteur** par le biais de son serveur web intégré, voir chapitre voir chapitre "Serveur Web", tant qu’aux opérations de modification d’un bloc **user** ou visualisation en temps réel les valeurs des variables en cours d’exécution (Running), elles ne sont disponibles que depuis l’**Éditeur de blocs**.

- 5.2 - Méthode d’exécution

Pour qu’un chaînage de blocs contenu dans un bloc **user** soient exécutable, il est impératif que ce chaînage se termine par un bloc <output> et que ce dernier soit associé à un événement (comme l’événement périodique à 100ms par exemple). l’**Exécuteur** évalue périodiquement tous les blocs <output> associés à un événement, déclenchant ainsi l’évaluation du bloc précédent qui nécessite évaluation du bloc précédent et ainsi de suite. Cette approche élimine la nécessité de définir explicitement l’ordre d’exécution.

Il est possible d’inclure dans un même blocs **user** des chaînes de blocs qui seront exécutés avec des périodicités différentes, en effet un bloc **user** peut contenir plusieurs blocs <output>, chaque <output> pouvant êtres associés à un événement périodique différent. Si il existe des liens entre les chaînes de blocs de périodicités différentes, il faudra insérer un bloc **system** <previous> entre chaque lien inter-chaîne, afin que la chaîne qui doit être exécuté lentement ne soit pas exécuté à la période de la chaîne la plus rapide.

- 5.3 - Chargement à la volée (hotswap)

Pour que le chargement à la volée soit possible, l’**Exécuteur** dispose pour chaque bloc **user** (programme) de deux versions **Shift A** et **Shift B**. La nouvelle version d’un bloc **user** est toujours téléchargée dans le **Shift** qui n’est pas en cours d’exécution, lorsqu’un « hot swap » est initié, les valeurs courantes des variables « mémorisées » du **Shift** en cours d’exécution sont transférés à l’autre **Shift**, qui prend alors la relève.

Les variables « mémorisées » sont des variables qui doivent être mémorisées entre deux cycles d’exécution se sont des sorties de bloc **system** tel que <previous>, <memory>, <clock>, <edge>, <integrator>…

- 5.4 - Bit de validité

De manière générale tant qu'un bloc n'a pas était évalué ses sorties sont « invalides »

La validité associée à chaque sortie est évaluée automatiquement lorsque le bloc est exécuté. Si une des entrées nécessaires à l'évaluation de la sortie est « invalide » ou si la sortie du bloc ne peut être évalué à cause d'un confit de type ou autre, la sortie sera « invalide », sinon elle sera « valide ».

- 5.5 - Interface IHM/SCADA

l'**Exécuteur** peut disposer d'un serveur OPC AU, des blocs **system** dédiés sont à votre disposition pour y exposer les variables que vous souhaitez.

- OPC_server : configure un serveur OPC UA
- OPC_node : crée un nœud
- POC_var : crée une variable
- OPC_read : lit une variable
- OPC_write : écrit une variable

- 5.6 - Entrées/Sorties déportées

Les « remote I/O » sont gérées par des blocs **system** dédiés.

Actuellement, seul le protocole Modbus/TCP est pris en charge via les blocs suivants:

- modbus_conn : connection TCP avec le module d'entrées sorties déportées
- modbus_read : lecture dans un esclave ModBus
- modbus_write : écriture dans un esclave ModBus

- 5.7 - Raspberry pi GPIO

Les pines GPIO sont gérées par des blocs **system** dédiés suivants:

- GPIO_DI : configuration & lecture d'une entrée digital d'une pine du GPIO
- GPIO_DO : configuration & écriture d'une sortie digital d'une pine du GPIO
- GPIO_PWM : configuration & écriture d'une sortie MLI d'une pine du GPIO

- 5.8 - Créer un nouveau bloc system

Pour ajouter un bloc à la bibliothèque standard, il faut :

1. Créer l'interface externe (interface graphique) du nouveau bloc **system**.
2. Codez la fonction « Python » associée au nouveau bloc **system** et ajouter le nom du bloc à la liste des blocs de la bibliothèque standard.

Notez qu'il faudra redémarrer l'**Exécuteur** et l'**Éditeur de blocs** pour prendre en compte ce nouveau bloc **system.**

- 5.8.1 - Créer l'interface externe

Ouvrir l'**Éditeur de blocs**.

Dans la barre de menu, cliquez sur « Bloc » puis « Change Properties », afin d'ouvrir la fenêtre « bloc info » suivante :

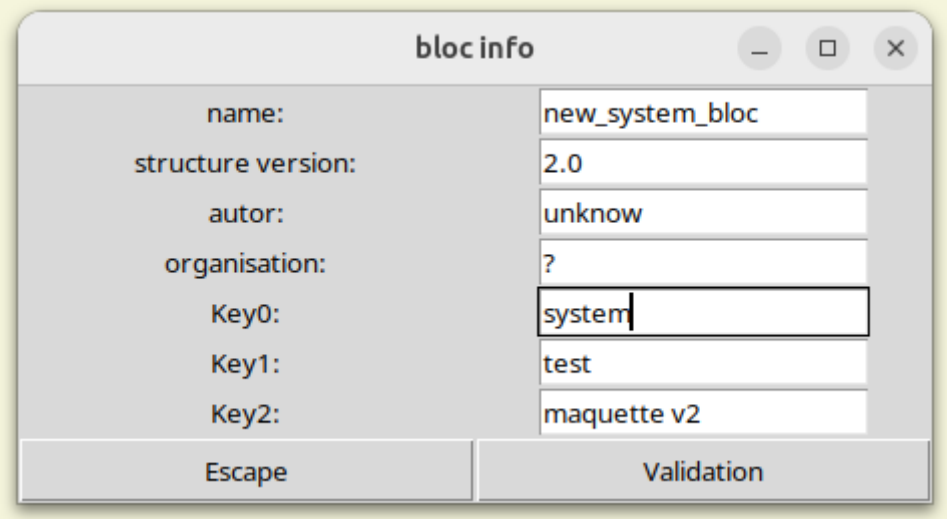


Figure : bloc info

Saisissez le nom du nouveau bloc **system** dans le champ « name: ».

Saisissez le mot-clé « system » dans le champ « Key0: ».

Confirmez votre saisie en cliquant sur « Vadidation »

Dans la zone graphique, ajoutez des blocs **system** <input> et/ou <output> selon le nombre d'entrées et de sorties nécessaires, renommez ces entrées/sorties, donnez une valeur par défaut aux entrées, définissez éventuellement certaines sorties en temps que variable « mémorisée » puis appuyez sur la touche [F3] pour sauvegardez l'interface graphique de votre nouveau bloc **system**.

Pour vérifier que l'interface a bien été créée, ouvrez une nouvelle fenêtre puis ajoutez le bloc **system** fraîchement créé, et vérifiez que l'interface externe correspond à ce que vous souhaitez.

- 5.8.2 - Créer le code Python

- 5.8.2.a - Créer la procédure Python liée au nouveau bloc system

Les fonctions associées aux bloc **system** sont dans le fichier /Target/exec.py, par exemple, voici la fonction Python associée au bloc **system** <and> :

```
1  def c_exesubloc_and (pebloc, pieb, pio, pthread):
2      """ exécution du bloc a et b (dans la boucle récurcive)"""
3      cesubloc = pebloc.sublocs[pieb]
4      #print ("<AND> les paramètres sont: pieb=", pieb, ",   pio=", pio, ",   counter=", pthread['counter'])
5      if cesubloc.header['counter'] == pthread['counter']:
6          pass
7          #print ("<AND>", "  cesubloc['counter'] == pthread['counter']: =", pthread['counter'], "   (output[", pio, "] inchangée)")
8      else:
9          cesubloc.header['counter'] = pthread['counter']
10         try:
11             pebloc.c_exebloc_recup_inputs(pieb, pthread)
12             cesubloc.c_exesubloc_validation_standard()
13             cesubloc.outputs[0]['var'] = cesubloc.inputs[0]['var'] and cesubloc.inputs[1]['var']
14         except:
15             print ("<AND>", PARAM_TEXT_EXCEPTION)
16             for output in cesubloc.outputs:
17                 output['valide'] = False
18         cesubloc.c_exesubloc_overwriting_outputs()
19     #print ("<AND> retourne l'output [", pio, "]: var=", cesubloc.outputs[pio]['var'], "val=", cesubloc.outputs[pio]['valide'])
20     return cesubloc.outputs[pio]
```

Figure: fonction « c_exesubloc_and »

Si on fait abstraction des éléments liés au débogage il ne reste que ceci :

```
1  def c_exesubloc_and (pebloc, pieb, pio, pttread):

3          cesubloc = pebloc.sublocs[pieb]

5          if cesubloc.header['counter'] == pthread[‘pcounter’]:

6                  pass

8          else:

9                  cesubloc.header['counter'] = pthread[‘pcounter’]

10                 try:

11                         pebloc.c_exebloc_recup_inputs(pieb, pthread)

12                         cesubloc.c_exesubloc_validation_standard()

13                         cesubloc.outputs[0]['var'] = cesubloc.inputs[0]['var'] and cesubloc.inputs[1]['var']

14                 except:

16                         for  output in cesubloc.outputs :

17                                 output['valide'] = False

18                 cesubloc.c_exesubloc_overwriting_outputs()

20         return cesubloc.outputs[pio]
```

Explication du code :

Ligne 1 : Le nom de la fonction est « c_exsubloc_ » suivi du nom du bloc system. Les paramètres sont :

- pebloc : est la structure « exécutable » du bloc user en cours d'exécution.
- pieb : est l'index du bloc system à exécuter
- pio : est l'index de la sortie dont il faut retourner la valeur
- pthread : est la structure contenant toutes les taches

Ligne 3 : permet de pointer sur le bloc à exécuter.

Ligne 5 : test si le bloc a déjà été exécuté

Ligne 9 : marque le bloc comme déjà exécuté

Ligne 11 : récupère toutes les entrées du bloc

Ligne 12: affecte tous les bits de validation des sorties du bloc en fonction de la validité des entrées

Ligne 13: affecte la sortie en fonction des entrées (ET logique)

Lignes 16&17 : affecte le bit de validité à « invalide » dans le cas où le calcul du bloc ne se soit pas passé correctement

Ligne 20 : retourne la sortie demandée (sortie d'index pio)

Dans la plupart des cas votre nouvelle fonction sera identique à celle de bloc system <and>, sauf :

- la ligne 1 : puisque le nom de la fonction sera différent.
- la ligne 13 : puisque vous affecterez les sorties en fonction des entrées selon votre besoin.

- 5.8.2.b - Associer la procédure au paramètre « nom de bloc »

Toujours dans le fichier: /Target/exec.py, il faut ajouter l'association entre le nom de votre nouveau bloc **system** et la fonction Python fraîchement créée, ceux-ci se passe dans la fonction « recup_procedure ».

Notez que cette fonction est aussi utilisée par l'**Éditeur de bloc**s lorsque qu'un bloc **user** est compilé :

```
def recup_procedure(psubloc):
    proc_name = "recupe_procedure"
    """ ajout l'adresse de la procédure qui correspond au nom du bloc"""
    if   psubloc.header['name'] == PARAM_NAME_BLOC_DT:          procedure= c_exesubloc_dt
    elif psubloc.header['name'] == PARAM_NAME_BLOC_OUTPUT:      procedure= c_exesubloc_output
    elif psubloc.header['name'] == PARAM_NAME_BLOC_INPUT:       procedure= c_exesubloc_input
    elif psubloc.header['name'] == PARAM_NAME_BLOC_PREVIOUS:    procedure= c_exesubloc_previous
    elif psubloc.header['name'] == PARAM_NAME_BLOC_SELECT:      procedure= c_exesubloc_select
    elif psubloc.header['name'] == PARAM_NAME_BLOC_VALIDREAD:   procedure= c_exesubloc_validRead
    elif psubloc.header['name'] == PARAM_NAME_BLOC_VALIDWRITE:  procedure= c_exesubloc_validWrite
    elif psubloc.header['name'] == PARAM_NAME_BLOC_COMP:        procedure= c_exesubloc_comp
    elif psubloc.header['name'] == PARAM_NAME_BLOC_AND:         procedure= c_exesubloc_and
    elif psubloc.header['name'] == PARAM_NAME_BLOC_OR:          procedure= c_exesubloc_or
    elif psubloc.header['name'] == PARAM_NAME_BLOC_NOT:         procedure= c_exesubloc_not
    elif psubloc.header['name'] == PARAM_NAME_BLOC_EDGE:        procedure= c_exesubloc_edge
    elif psubloc.header['name'] == PARAM_NAME_BLOC_CABLIN:      procedure= c_exesubloc_cablin
    elif psubloc.header['name'] == PARAM_NAME_BLOC_CABLOUT:     procedure= c_exesubloc_cablout
    elif psubloc.header['name'] == PARAM_NAME_BLOC_ADD:         procedure= c_exesubloc_add
    elif psubloc.header['name'] == PARAM_NAME_BLOC_SUB:         procedure= c_exesubloc_sub
    elif psubloc.header['name'] == PARAM_NAME_BLOC_MULT:        procedure= c_exesubloc_mult
    elif psubloc.header['name'] == PARAM_NAME_BLOC_DIV:         procedure= c_exesubloc_div
    elif psubloc.header['name'] == PARAM_NAME_BLOC_MINMAX:      procedure= c_exesubloc_minmax
    elif psubloc.header['name'] == PARAM_NAME_BLOC_CONST_PI:    procedure= c_exesubloc_const_pi
    else:
        print (proc_name, " ERROR: function not defined for this bloc <"+psubloc.header['name']+">")
    return procedure
```

Figure : fonction « recup_procedure »

Pour cela vous devrez ajouter une ligne avant le « else » est créer un nouveau paramètre dont le nom sera constitué de :

« PARAM_NAME_BLOC_ » suivi du nom de votre nouveau bloc **system** en majuscule.

- 5.8.2.c - Définir le nom de fichier du nouveau bloc system

Il faudra aussi modifier le fichier /Target/PARAM_NAME_BLOC.py, en y ajouter une ligne pour définir la constante « PARARM_NAME_BLOC_*votrenomdebloc* », la chaine de caractère située à droite du signe « = » correspond au nom de ficher votre nouveau bloc **system** dont l'extension sera toujours « .bloc ».

```
# ATTENTION: la source de ce fichier ce trouve dans le répertoire "Target"

# nom des blocs "system"
PARAM_NAME_BLOC_DT = "dt"
PARAM_NAME_BLOC_OUTPUT = "output"
PARAM_NAME_BLOC_INPUT = "input"
PARAM_NAME_BLOC_PREVIOUS = "previous"
PARAM_NAME_BLOC_SELECT = "select"
PARAM_NAME_BLOC_VALIDREAD = "validRead"
PARAM_NAME_BLOC_VALIDWRITE = "validWrite"
PARAM_NAME_BLOC_MINMAX = "minmax"
PARAM_NAME_BLOC_COMP = "comp"
PARAM_NAME_BLOC_AND = "and"
PARAM_NAME_BLOC_OR = "or"
PARAM_NAME_BLOC_NOT = "not"
PARAM_NAME_BLOC_EDGE = "edge"
PARAM_NAME_BLOC_ADD = "add"
PARAM_NAME_BLOC_SUB = "sub"
PARAM_NAME_BLOC_MULT = "mult"
PARAM_NAME_BLOC_DIV = "div"
PARAM_NAME_BLOC_CABLIN = "cablin"
PARAM_NAME_BLOC_CABLOUT = "cablout"
PARAM_NAME_BLOC_CONST_PI = "const.pi"

```

Figure : fichier PARAM_NAME_BLOC.py

- 5.9 - Startup (autorun)

Les blocs de votre choix peuvent être démarrés automatiquement lorsque l'**Exécuteur** est lancé, pour cela il faut configurer le « Startup ».

Principe :

- A chaque fois que vous transférez un bloc **user** de l'**Éditeur de blocs** vers l'**Exécuteur** un copie est faite dans la machine cible sous la forme d'un fichier dans le répertoire « eblocs\build » nom du fichier est constitué du nom du bloc suivi de date et l'heure de sa compilation effectuée par l'**Éditeur de blocs.**
- Le répertoire « eblocs\build » contient tous les blocs susceptibles d'être ajoutés au « Startup » .
- Le répertoire « eblocs\startup » contient tous les blocs à exécuter lorsque l'**Exécuteur** est lancé.

- 5.9.1 - Ajouter un bloc dans le « startup »

il suffit de déplacer le fichier correspondant du répertoire « eblocs\build » vers « eblocs\startup ».

- 5.9.2 - Retirer un bloc du « startup »

il suffit de déplacer le fichier correspondant du répertoire « eblocs\startup » vers « eblocs\build ».

- 5.9.3 - Supprimer définitivement un bloc

il suffit de supprimer le fichier correspondant du répertoire « eblocs\build » ou « eblocs\startup ».

Vous pouvez effectuer ces opérations de déplacement/suppression de fichier à l'aide de votre « explorateur de fichier » (file manager), ou à travers les « pages web » fournis par la cible, en sélectionnant « Startup » dans le menu principal.

- 5.10 - Serveur Web

- 5.10.1 - Généralités

Un serveur HTTP est intégré à l'**Exécuteur,** il permet à l'utilisateur par le biais d'un simple navigateur web de visualiser l'état de la Machine Cible :

- Visualiser les taches disponibles et le nombre de <output> associé
- Visualiser la charge CPU générale et par tache
- Visualiser la liste des blocs disponible dans la Machine Cible et l'état (Pending, Ready, Running) des **Shift**s **(**A & B)
- Visualiser l'état et la valeur des <output> de tous les blocs pour les **Shift**s (A & B)
- Visualisé les connexions Ethernet en cours gérées par l'**Exécuteur**

Il permet aussi de passer des commandes simples telle que :

- Arrêter l'exécution d'un bloc **user** (commande : Stop)
- Démarrer l'exécution d'un bloc **user** (commande : Run)
- Initialiser un bloc **user** (commande : Initialize)
- Supprimer un bloc **user** (commande : Delete)
- Intervertir les **Shift A** et **Shift B**
  - lorsqu'aucun **Shift** est « Running » (commande : ClodSwap)
  - lorsqu'un **Shift** est « Running » (commande : HotSwap)
- Configurer les blocs qui seront lancés automatiquement au démarrage de l'**Exécuteur** (srtatup)
- Lancer des commandes dédiés au débug
  - Impression de tous blocs connus par l'**Exécuteur** (commande : print list_compiled)
  - Impression de toutes les tâches connues par l'**Exécuteur** (commande : print list_threads)

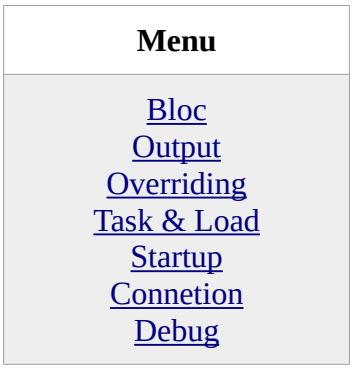
- 5.10.2 - Adresse du serveur web

Par défaut l'adresse est: http://127.0.0.1:55555/

Comme dans la version d'évaluation (téléchargeable sur Github) le PC cible et le PC de développement sont sur le même PC l'adresse IP du serveur web est 127,0,0,1, (notez le port 80 qui est le port par défaut pour le protocole HTTP n'est pas utilisé) nous utilisons le port 55555 afin de pouvoir lancer l'**Exécuteur** sans avoir besoin d'une commande « sudo ».

- 5.10.3 - Menu principal

Target web page:192.168.5.58/



- **Bloc :** liste les blocs connus par l’ **Exécuteur**, en indique le status et propose les ordres à exécuter en fonction de l’état.
- **Output :** liste tous les outputs des blocs connu par l’ **Exécuteur**, en indique la valeur et la validité
- **Overriding :** liste tous les forçages connus par l’ **Exécuteur**, en indique la valeur et la validité
- **Task & load :** liste toutes les taches connues par l’ **Exécuteur** , en indique le temps de tache
- **Startup :** liste les blocs qui sont configurés pour démarrés dés le lancement de l’ **Exécuteur,** liste aussi les blocs qui peuvent y être ajoutés.
- **Connection :** liste les serveurs et les clients connectés à l’ **Exécuteur**
- **Debug :** Donne accès à des outils de débogage

- 5.10.4 - Exemples de page web

- *5.10.4.a* - Page « Bloc »

Target web page:192.168.5.58/blocs

bloc list					
bloc name	**Shift A**		**Shift B**		**order**
	Status	**Build time** yyyy-mm-dd hh:mm:ss	**Status**	**Build time** yyyy-mm-dd hh:mm:ss	
loop	Running	2026-08-31 17:15:05	...	...	Stop Initialize

Refresh Main menu

- 5.10.4.b - Page « Output »

Target web page:192.168.5.58/outputs

bloc output list										
bloc			**task**			**output**				
name	**shift**	**status**	**name**	**id**	**exec order**	**name**	**id**	**type**	**validity**	**value**
loop	A	Running	1s	2	0	Oloop	4	int	😊	2213

Show only Running shifts

Refresh Main menu

- 5.10.4.c - Page « Overriding »

Target web page:192.168.5.58/overwriting

Overwriting list							
bloc			**io**				
main	**shift**	**path/name**	**name**	**type**	**id**	**value**	**validity**
loop	B	loop/(3)previous	var1	output	3	2566	😊

Refresh Main menu

- 5.10.4.d - Page « Task & loads »

Target web page:192.168.5.58/threads

Task list								
Setting			Feedback					
name	id	period	cycle time	cycle time min	cycle time max	counter	number of output	duration
10s	0	10s	10.000027s	9.999933s	10.000027s	299	0	0.004ms
3s	1	3s	3.000059s	2.999973s	3.000060s	998	0	0.007ms
1s	2	1s	1.000059s	0.999893s	1.000091s	2995	1	0.038ms
200ms	3	200ms	200.083ms	199.830ms	200.311ms	14976	0	0.006ms
100ms	4	100ms	100.014ms	99.814ms	101.264ms	29950	0	0.003ms
50ms	5	50ms	49.899ms	49.800ms	50.835ms	59899	0	0.002ms
20ms	6	20ms	20.040ms	19.780ms	20.937ms	149769	0	0.005ms
10ms	7	10ms	9.889ms	9.783ms	10.850ms	299604	0	0.004ms

Refresh Main menu

- 5.10.4.e - Page « Startup »

Target web page:192.168.5.58/startup

Actual Startup configuration	
name	**order**
loop__2026-08-31__17:15:05.ebloc	Remove

Blocs available for Startup	
name	**order**
ex_cable_2__2026-08-31__17:52:18.ebloc	Add Delete
dent_de_scie__2026-08-31__21:50:58.ebloc	Add Delete
loop__2026-09-02__11:23:27.ebloc	Add Delete

Refresh Main menu

- 5.10.4.f - Page « Connection »

Target web page:192.168.5.58/connetion

HTTP protocol			
...	**@ip**	**port**	**Host name**
Server	192.168.5.58	55555	jbar-HLYL-WXX9
Last Client	192.168.5.58	38498	jbar-HLYL-WXX9

TCP/IP protocol		
...	**@ip**	**port**
server	192.168.5.58	55556
client[0]	127.0.0.1	36628

Refresh Main menu

- 5.10.4.g - Page « Debug »

Target web page:192.168.5.58/debug

debugging tools
print list_compiled
print list_threads

Refresh Main menu